

第 2 次评估 · 过程性评估

# A2 — 应用人工智能系统

小组作业 · 占课程成绩的 20% · 由教师评分

|      |                                 |       |              |
|------|---------------------------------|-------|--------------|
| 发布日期 | 2026 年 8 月 30 日（星期日）            | 权重    | 20%          |
| 截止日期 | 2026 年 9 月 13 日，星期日，新加坡时间 23:59 | 涵盖内容  | 第 4 – 6 周    |
| 团队声明 | 2026 年 9 月 4 日，星期五，23:59（新加坡时间） | 评分标准  | 评分标准 1（四个标准） |
| 提交至  | NTULearn                        | Teams | 您现有的课程小组     |

用一段话来解释 A2 是什么。第四节课认为，集成方案才是最终交付物——智能体本身是相对简单的一部分，真正将一个可以部署的系统与一次偶然成功的演示区分开来的，是其周边的证据。A2 证明了这一观点——而且并不仅限于第四节课。A2 贯穿第 4、5、6 周，每节课都提取了特定内容。第四节课提供了：循环、工具层、护栏机制、评估集。第五节课提供了：三层成本模型（按任务变量、预期降级成本、固定月成本），并根据您实际问题的数据量定价，包括敏感性分析范围而非单一估计值，以及廉价模型需要达到的盈亏平衡成功率。第六节课提供了：护栏检查清单和威胁暴露面，因为这些问题都涉及读取来自您组织外部书写的内容。

如果你是那三分之二非技术背景的同学，请先阅读以下内容。每个团队都至少有一名成员负责编程，这也是特意安排的——但这并不意味着只有程序员做 A2，其他所有人都在一旁观看。请看分数占比：概念理解（25%）和推理与论证（25%）合计占总分的一半，而这些内容几乎都不涉及代码。D0 是写作任务。在 D2 中选择工具集并撰写六个字段的描述属于设计环节。编写评估用例属于领域判断——即知道哪些声明应当被拒绝。阅读成本模型并指出哪个杠杆因素最为关键属于商业分析。一个只有程序员在写代码、其他人完全不参与的团队，可能会实现一个能工作的智能体，但最终提交的材料将会很薄弱。

请将这三周的内容视为一个整体论点。第四节课问的是“它是否有效？”，第五节课问的是“当它失效时会产生什么成本？”，第六节课问的是“当有人攻击它时会发生什么？”。你的提交材料必须针对同一个系统回答这三个问题。

一个功能有限但配有真实评估数据集、真实的约束条件、经过防御的自主性设定以及诚实的成本模型的智能体，比起一个看似惊艳而无任何证据支持的智能体，将获得更高的分数。

## 本简报的内容包括

第 1 至 3 部分是需要你完成的内容。第 5 部分是评分标准——请在开始前阅读。附录 A 包含了全部两个问题。

|                      |    |
|----------------------|----|
| 1 · 选择你的问题           | 2  |
| 2 · 我们提供什么，以及何时提供    | 2  |
| 3 · 你需要构建的内容         | 4  |
| D0 · 为什么需要一个智能体      | 4  |
| D1 · 智能体             | 6  |
| D2 · 工具层             | 8  |
| D3 · 安全护栏层以及安全护栏检查清单 | 10 |
| D4 · 评估集             | 11 |
| D5 · 模型电池            | 11 |
| D6 · 服务成本模型          | 11 |
| D7 · 两种重现的故障         | 14 |

|                    |    |
|--------------------|----|
| 4 · 你需要提交的内容       | 15 |
| 5 · A2 的评分方式       | 15 |
| 6 · 利用 AI 构建 AI    | 16 |
| 7 · 预算             | 16 |
| 8 · 团队、贡献与搭便车条款    | 17 |
| 9 · 你何时会收到反馈?      | 18 |
| 10 · 时间表           | 18 |
| 11 · 此简报中真正起作用的关键词 | 19 |
| 问题 A · 健康保险理赔的首次回应 | 21 |
| 问题 B · 门诊转诊协调      | 23 |

1 · 选择你的问题

A2 不需要你定义问题。请在两个问题中任选其一。（期末项目允许你自行定义问题，而 A2 则固定了问题内容，以确保评分标准专注于你的构建过程和论证能力。）

两个问题具有相同的结构，该结构与第四节课所教授的模式一致：一个请求到达后，代理必须先查询相关信息，然后得出结论，最终达到三种结果之一——执行操作、请求缺失的信息，或转交给人工处理。两个问题都包含执行操作可能出错的情况，且两种情况下所需的步骤数取决于代理检索到的信息内容。

|   | 问题         | 领域     | 带门控的动作 |
|---|------------|--------|--------|
| A | 健康保险理赔首次响应 | 保险业务操作 | 签发决定函  |
| B | 门诊转诊协调     | 医院运作   | 预约诊所就诊 |

完整题目请见附录 A。请在选择前通读两题。两题难度相当，并且都需要真正使用循环；它们的难点不同——问题 A 在于单个索赔可能涉及的检查数量庞大，问题 B 则在于误操作时的高昂代价。

不可选择退出，也不支持自带问题。如果您希望针对自己的题目进行开发，这正是课程项目的目的所在。

A2 不得基于任何学生的课程项目进行开发

- 你不能将任何成员的个人项目内容作为 A2 任务的一部分进行提交，包括系统、数据集、评估集或报告。
- 你之后也不能将 A2 任务的作品——包括智能体、工具框架、测试用例和报告——作为你个人项目的一部分进行提交。
- 技术可以自由借鉴，但成果不得重复使用。任何内容都不允许被标记两次。

如果你的项目构想与 A2 的两个问题之一相似，这没有问题，且不构成冲突——但两者必须作为独立的工作分别完成，且你的项目必须始终是你个人独立提交的内容。

2 · 我们提供什么，以及何时提供

| 什么                                                                          | 何时                            |
|-----------------------------------------------------------------------------|-------------------------------|
| 本简介和常见问题解答 (FAQ)                                                            | NTULearn，自发布之日起 — 8 月 30 日星期日 |
| 团队自评表                                                                       | NTULearn，发布之日起——8 月 30 日（周日）  |
| 设备数据生成器——为你的问题生成初始数据文件，以免团队在第一个晚上花费时间来发明记录                                  | NTULearn，9 月 2 日（周三）中午前       |
| 启动用的代码笔记——无供应商锁定的配置，已将第 4 类循环扩展为每回合可解析多个动作（D2c），工具合约模板，用于填充的评估框架骨架，以及成本计算单元 | NTULearn，截至 9 月 2 日（周三）中午前    |
| 已归你所有——第 4 类胶囊 2 的代码笔记是一个具备五种工具、安全护栏和评估框架的可运行智能体。它只是一个骨架，从它开始是预期的行为，不会被扣分。  | 你已拥有                          |

脚手架只是一个起点，而不是一份规范。它的存在是为了避免任何人花时间在设置代码上。你可以随意修改其中的任何内容——重命名、重组结构、替换整个部分，甚至舍弃某些内容，如果你愿意，也可以从零开始自行编写。评分标准不会因为保留我们的代码而给予奖励，也不会因为偏离它而扣分。唯一重要的是你的系统要符合本任务书中的 A2 目标。如果脚手架的结构与你的问题相冲突，那么是脚手架有问题，而不是你的问题。

夹具数据只是一个种子，而非固定的集合，可以对其进行扩展。生成器会编写一组初始记录，更重要的是，它会向您展示模式：每个文件包含什么内容，字段如何命名，以及文件之间如何关联。

最后一个最重要，因为它决定了你的代理如何从一个事实推导到下一个事实。在问题 A 中，一个索赔记录包含 `member_id`，而成员文件则以相同的 `member_id` 作为键—这使得 `lookup_policy` 能够获取 `get_claim` 返回的 `member_id`，找到对应的成员，并通过其 `policy_id` 追踪到覆盖该成员的保险单。虚构一个保险索赔，其

`member_id` 与任何人不匹配，你的代理将完全正常并返回空值。

根据以下模式自由添加你自己的记录。你需要准备一个包含 30 – 50 个案例的评估集，其中有 6 – 10 个负面案例，这些记录要能触发这些负面结果，而有意地创造这些情况正是这项工作的一部分。只有两个条件：保留我们提供的记录（这样一个标记就会用这些记录重新运行你的测试工具），并且提交任何用来生成或保存新增记录的内容，以确保数据集是可复现的，而不是一个谜团。我们提供的记录附带有带标签的答案键。对于起始集中的每一条记录，`expected_outcomes_A.json` / `_B.json` 文件会给出附录 A 中的路由表所要求的结果以及产生该结果的单一触发条件。这就是你 D4 任务的基准事实，并且你自行创建的案例也必须采用这种格式进行标注。如果一个运行虽然达到了正确结果，但使用了错误的触发器，将不视为通过—它只是幸运地达到了结果，下次运行将无法再达到同样结果。

发现错误？请告诉我，这算数

生成器和脚手架是为本次作业编写的，并未经过上百人的修改。如果您发现任何缺陷——例如相互矛盾的数据、从未被填充的字段、功能与自身描述不符、无法满足的条件，或是本说明中的指令与其他部分相矛盾，请及时报告。

直接发送邮件给我，且越早越好。如果在 9 月 2 日发现的缺陷，能帮助到其他 27 支队伍；但如果直到 9 月 12 日才报告，对任何人都无益，包括您自己。

我承认的缺陷将在课堂参与评分中给予认可。这并非形式主义：注意到一个系统的文档与其行为不一致，恰恰是本课程花费六周时间论述的核心技能，也是形成良好评估集的重要直觉。请报告您发现了什么、在哪里发现的，以及您原本期望得到什么——这与任何地方撰写优秀 bug 报告的格式一致。

您无需等待修复。在您的代码仓库中记录下发现的问题、您做出的替代假设，并继续前进。一个文档化了的 workaround 是一个强有力的解决方案，而不是一个妥协的方案。

你直到周一之前都不会被限制。D0 的工作是写作，并非编程，而 D2(a) 和 D2(b) 中的工具集和描述符属于设计部分。请从这些开始——这正是本任务所要求的顺序。

你的 10 美元 OpenRouter 密钥是你已持有的那个。本次作业 A2 不会提供额外积分。

3 · 你要构建的内容 —— 八个必需部分

以下是必须完成的所有内容。顺序即为构建的顺序。

D0 · 为什么要使用智能体——在你写任何代码之前回答

这是我们要阅读的第一件事，也是你报告的第一部分。四类课程以它开始，它的学习笔记也以它开始。任何跳过它而直接进入构建的团队，已经在两个评分标准上失分了。

D0(a) · 将你的问题放在四类的阶梯上，并论证你的台阶

四类知识胶囊 1 在黑板上列出了七个台阶，从 Anthropic 的预读材料 1 中提取而来，按成本从低到高排列。第一至第六个台阶是工作流程——可预测、可枚举、可测试。只有第七个台阶才会将任务序列交给模型。

| 台阶            | 谁来决定?   | 成本                              | 何时使用               |
|---------------|---------|---------------------------------|--------------------|
| 1 · 单次调用      | 你       | 1 调用                            | 一次提示，明确指定的任务       |
| 2 · 提示链       | 你       | 2 – 4 次呼叫                       | 固定子任务并在其间进行检查      |
| 3 · 路由        | 模型选择车道  | 1 + 1 呼叫                        | 不同的类别；简单的任务使用低成本模型 |
| 4 · 并行化       | 你       | N 次调用，一个时钟                      | 独立检查，或是一次投票        |
| 5 · 调度器-工作者   | 模型，在运行时 | 不可预测                            | 划分取决于输入            |
| 6 · 评估者 – 优化者 | 模型      | 未知轮次                            | 标准可陈述；进行第二次遍历有帮助   |
| 7 · 智能体       | 模型      | 在你未设限之前，它没有边界步骤无法提前知晓<br>它会一直持续 |                    |

第四类规则不变：只有当上一级确实无法完成任务时，才继续向下一级。A2 位于第七级。你的任务不是为选择它辩护——是任务简报选择了它。你的任务是说明第一至第六级在你的问题上能提供什么、不能提供什么，以及第七级给你带来了什么代价。

以书面形式回答这些问题。

1. workflow 测试。四个问题可以区分 workflow 和智能体：第一个问题

|                  | workflow     | 智能体              |
|------------------|--------------|------------------|
| 谁来决定步骤的顺序以及何时执行? | 在代码中事先由你决定   | 由模型在执行时即时决定      |
| 步数是否随输入而变化?      | 否——每次都走相同的路径 | 是——这正是关键所在       |
| 你能测试所有路径吗?       | 是的，它们是可枚举的   | 不——你测试的是结果，而不是路径 |
| 成本是多少?           | 可预测：n 次调用    | 直到你把它封顶，否则是不可预测的 |

第二行是决定性的。如果步数不随输入变化，那你就需要一个 workflow——不管听起来有多巧妙。请用你自己评估集中的例子证明，你的方法确实有所变化。

2 · 两种条件。一个智能体是一个系统，其中模型“动态地调度其自身的过程和工具使用，保持对其完成任务方式的控制。”必须同时满足两个条件——缺少其中一个，主体就不再是原来的东西了：

- 这些步骤事先未知。如果没有这一点，你就只拥有一 workflow。
- 它在每个步骤中都能获得真实情况的反馈 —— 比如工具执行的结果、查询内容或实际运行的代码 —— 因此现实世界可以对它进行纠正。如果没有这一点，你就只剩下一个独白：虽然流畅、自治，但无法被证伪。

3 · 三问测试 —— 管理悬崖所在之处。谁来选择检索的内容?

|          |               | 它能否循环并重新查询?       | 它能改变世界吗? |
|----------|---------------|-------------------|----------|
| 检索 (RAG) | 你的代码——一个固定的查询 | 否——只执行一次          | 否——只读    |
| 代理式检索    | 模型在运行时        | 是——它会根据所找到的内容再次查询 | 否——只读    |

|    | 谁决定检索什么内容？ | 它能否循环并重新查询？ | 它能否改变世界？          |
|----|------------|-------------|-------------------|
| 代理 | 模型，在运行时    | 是           | 是 — 它会编写、发送、预订并付款 |

代理式检索实际上只是一个工具均为只读的代理。治理悬崖并非检索 → 代理式检索，而是代理式检索 → 代理，在第一次写入操作时发生。为你的第一个无法逆转的操作命名；正是那次写入行为决定了你的问题处于第 7 层而非第 5 或第 6 层。

D0(b) · 何时不应构建代理——回答胶囊 1 中的两个测试

胶囊 1 提供了两个测试，并表示若某任务任一测试不通过，就应将其构建在较低层级。请回答两者。

测试 1 · 真实性测试。什么将告诉循环过程它正在出错，以及多快？一套失败的测试套件能在几秒内给出答案且无法辩驳；而客户满意度的反馈则需数天且取决于个人意见。快速且客观 → 应构建代理，并首先将该信号集成进去。若反馈缓慢或带有主观性 → 则应采用包含人工审核环节的工作流。而如果根本没有任何反馈信号，你就根本没有 AI 问题，而是一个未定义的问题——无论自主性如何都无法解决一个无人能明确表述的目标。

列出你的问题中可能在几秒钟内与模型产生冲突的记录系统——策略行、槽位表、文档状态——并说明每个系统的响应速度。第 4 组给出的有效答案是一个订单行、一个承运商扫描和一个库存盘点：以机器的速度提供真实数据，这正是允许循环无人监管运行的全部原因。

测试 2 · 数学计算——这个测试结果会影响后面的所有任务。

请仔细阅读指数部分：它指的是单次运行中的步骤数，而不是你的评估集中的案例数量。第 4 组提出了一个情况：当一条单一的轨迹需要连续正确完成二十项任务时会发生什么。这里和你测试了多少案例无关。

每一步的可靠性会随着一次运行的进行而累积。如果每一步的成功率是 95%，而一次运行总共需要 20 个步骤：

|             |   |      |                        |
|-------------|---|------|------------------------|
| $0.95^{20}$ | ~ | 0.36 | 一次运行当中，每一步都正确的概率为三分之一。 |
| $0.99^{20}$ | ~ | 0.82 | 即使每一步的成功率有 99%。        |

笔记一提出了三种解决方案，中间的方法是最大的突破：提高单步的可靠性（最难） · 减少步骤数量（最大的收益） · 使错误可恢复。

现在用你自己的数据来算一下——并注意你正朝着哪个方向努力。你无法直接测量单步的可靠性；你在 D4 中测量的是整个运行是否产生了正确结果。因此，你需要反向思考。如果你测得的运行通过率是 P，从 D7 测得的中位回合数是 T，那么产生这一结果的单步可靠性应为：

|                            |                              |                    |
|----------------------------|------------------------------|--------------------|
| 每步隐含的可靠性                   | $s = P^{(1/T)}$              |                    |
| 示例：                        | $P = 0.78$ 在中位数 $T = 6$ 轮中测得 |                    |
| $s = 0.78^{(1/6)} = 0.959$ |                              | 每一步大约有 96% 的概率向右移动 |
| 在 $T = 3$ 时 ->             | $0.959^3 = 0.88$             | 切割到三个回合预测 ~ 88%    |
| 在 $T = 12$ 时 ->            | $0.959^{12} = 0.61$          | 徘徊至十二步预测 ~ 61%     |

这就是整个论点的三点总结：同一个代理，每一步的质量相同——然而运行成功率却仅凭回合数的不同，在 61%到 88%之间浮动。第二种思路是 D2(c)——并行的工具调用会压缩回合数——你刚刚调整过的运行通过率，正好决定了 D6 中成本模型的第二层，因为第二层是  $(1 - P) \times$  失败成本。这三个并非三个独立的实验。它们是一个完整论证的三种测量方式。

并非每个步骤的 s 都相同——找到薄弱环节正是该实验的目的。读取策略行的回合近乎完美。而评估成员自由文本叙述的回合则不尽人意。用单一数值来平均两者会掩盖彼此之间的差异。

你的追踪日志已经包含了答案。按导致失败前最后调用的是哪个工具，来分组你那些失败的运行。在错误结果之前出现次数最多的回合，就是你要寻找的目标。

接着，注意你这里实际上有两种不同的操作，而它们并非同一类杠杆。

|     | 这一举动                                      | 属于第 4 类三种方式中的哪一种 | 它改变了什么          |
|-----|-------------------------------------------|------------------|-----------------|
| (a) | 修正该步骤——使用更优的工具描述符、过滤后的返回结果以及更严格的类型        | 方法一 · 提升每一步的可靠性  | s 增大，因此 P 也随之增大 |
| (b) | 移除或绕过该步骤——切断工具，继续操作<br>将其转换为普通代码，或折叠为并行循环 | 方法二 · 减少步骤数量     | T 下降，P 就会上升     |

这里有一个值得写进报告中的联系：一个薄弱的步骤通常会让你两者都付出代价。当某一步返回了糟糕的结果时，智能体倾向于重新阅读、重试或者徘徊不前——因此一个不可靠的步骤同时会降低 s 并增加 T。这就是为什么找到它能带来回报：修复一处即可同时改善两个指标。这也解释了为什么你在 D7 中的循环失败和这个数学关系，其实是从两个不同角度观察的同一项调查。

说明限制条件，因为这些是实际存在的。步骤之间并不相互独立——早期的不良观察会使得后续步骤更差，成功概率不一；而且并非所有步骤的故障率都相同；例如，读取政策文件的工具调用比评判自由文本叙述的工具调用更可靠。因此，将 s 视为一种诊断工具，而不是你系统中的一个物理恒定参数。这是一种在询问“你的问题是步骤质量不足，还是步骤数量过多？”的方法，而答案将决定你应该重点改进哪一个。在报告中要明确说明你的证据指向哪一种情况。

D0(c) · 什么是“良好”的表现 —— 在任何代理代码编写之前

关于一次良好运行的五个编号陈述，在您首次提交代理之前已提交到仓库中（我们将查看提交历史）。第 4 类示例的格式如下：

- 1 · 指出了可追溯至记录的真实原因，而非一个看似合理的说法。
- 2 · 给出的结果与记录中的实际内容一致。
- 3 · 最多只执行一次门控操作，并且只在事实确立之后进行。
- 4. 如果没有记录支持，宁愿说“我不知道”，也不应自行编造答案。
- 5. 成本低于由人工完成。

第 4 项是团队最容易忘记的，也正是你的负面案例所要捕捉的地方。你在 D4 中的评估集是基于此项之后的——如果此处的陈述无法验证，请重写直到可测试为止。

D1 · 代理——以及你被要求构建的架构

A2 是一个基于单智能体 ReAct（推理 + 行动）的架构构建。这就是其体系结构，不是你可以选择禁用的默认设置。

一个智能体。一个控制循环。思考 → 行动 → 观察 → 重复 → 结束。这是一个在基础模型上运行的工具使用循环：它读取请求，决定要调用哪个或哪些工具，读取结果，再次决定，最后得出一个结果。班级 4 的笔记本提供了一个可立即使用的框架，你可以以此作为起点，而从它开始并不会受到惩罚。

你的智能体必须使用多个工具，并且在单个回合中必须能够调用超过一个工具。班级 4 的循环故意设计为每个回合只读取一个“行动”；而 A2 要求你扩展这一点。为什么以及怎么扩展，则由 D2(c) 说明。在一个回合中调用三个工具仍然是单个智能体——它并不属于多智能体，D2(c) 中的内容不会改变基础架构。

超出范围——这是一个范围限制，而非难度限制

| 不在 A2                                        | 原因                                                      |
|----------------------------------------------|---------------------------------------------------------|
| 多智能体系统——<br>协调者与工作者，调用其他智能体的智能体，专业智能体之间的任务交接 | 第 4 课第 1 个胶囊对比了这些模式。第 2 个胶囊只构建了其中一种。我们不会标记我们没有教过你的构建内容。 |
| 将子代理作为扩展工具集的方法                               | 原因相同。这些内容在摘要中列出，但从不实际构建，否则会隐藏 A2 所标记的循环。                |

| 不在 A2                                                      | 原因                                                                                                                                 |
|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| 拥有循环的框架——<br>LangChain、LangGraph、CrewAI、<br>AutoGen 及其相关项目 | 第 4 节课特意手工编写了循环，并称之为“框架陷阱”。所标记的东西正是这个循环。如果某个库替你运行循环，那么在 D2(c) 或护栏层中就没有任何你自己的东西可供标记了。而对于其他一切事务（如数据处理、HTTP、测试、绘图）的普通库则没有问题，并且是预期之中的。 |
| 微调模型                                                       | 这个内容没有在课程中教授，\$10 也无法实现，也不是本次作业的重点。                                                                                                |

但这并不禁止大家思考这些问题。恰恰相反，你们在报告中讨论这些问题会获得分数。LOI（预期学习成果）3 是关于评估替代设计方案的，所以写一段话，例如“如果有一个代理对我们的决定函进行二次审查，就能发现失败点 X——这里是我们项目因此付出的成本，以及我们为何没有选择这样构建”，正是我们希望看到的论证方式。预读材料 5 为此提供了证据：Cognition 在一个多代理的背景下推出 Devin，随后又发表了自我纠正文章——即《不要构建多代理系统》（2025 年 6 月）和《多代理系统：哪些真的管用》（2026 年 4 月），在这两篇文章中，审查代理本身是多代理系统，但其撰写内容仍然是单一主线的。

简而言之：构建一个 ReAct 代理即可。用叙述性文字来论述替代方案的选择。拿分的是论述过程，而不是未实现的架构。

范围边界 — 在开始任何计划工作之前请先阅读此部分

你的工具操作的是夹具数据：即在你本地机器上的小型文件——JSON（JavaScript 对象表示法）、CSV（逗号分隔值，可在 Excel 中打开），或者如果你团队中的某人已经使用该方式，则可以使用小型 SQLite 数据库文件。不连接任何真实系统，也不使用真实账户——你的代理所操作的任何内容都不应是任何人的实际记录。

能够访问公共互联网的工具——可选且有条件。你并不需要这样的工具。这里的所有问题都可以仅从其自身记录中回答，没有任何题目依赖于拥有一个搜索工具，而且最简化的、可以辩护的工具清单中几乎从不包含这类工具。但如果你希望包含一个互联网访问的工具，那么有一个条件是不可协商的：你的脚本执行流程仍必须在无网络和无密钥的情况下重现。调用一次该服务，将返回结果保存为夹具数据，随后让脚本后端重复回放这些结果。

这是针对任何与网络通信的内容进行测试的常规做法，也正是 D5(a)之所以能成为一个要求的唯一原因。一个因为网络内容发生变化而无法再次运行的评估集，并不是一个真正的评估集——同样的案例在周二可能通过而在周四失败，而这与你的代理本身毫无关系。选择用回放记录，或者直接舍弃。

受控动作是一个日志条目，而非其所代表的事物本身

这是团队在 A2 中浪费整整一周时间的最常见方式，因此需要明确指出。受控动作包含一个函数，三个步骤：（1）检查门控状态——自主性设置是否已满足，以及操作员是否在你的设置要求下进行了确认；（2）向本地文件追加一条结构化记录；（3）返回一个确认字符串，供代理将其写入最终答案。这就是全部写入操作。

具备这种结构的实现就是满分实施方案。

```
{
  "ts": "2026-09-08T14:22:07",
  "case_id": "CLM-8842",
  "decision": "原则批准",
  "reason": "保单有效期至 2027-03-31；包含全部 3 项保障内容；指定医院",
  "证据": [
    "获取理赔",
    "查找保单",
    "检查保障范围 x3",
    "获取预先授权",
    "获取医院状态"
  ],
  "自主性": "确认",
  "闸门": "操作员在第 4 回合批准",
  "turns": 4,
  "cost_usd": 0.0041
}
```

你不需要构建的内容。以下任何一项都不会获得任何分数，并且每项都会占用你的时间：

|         |             |           |
|---------|-------------|-----------|
| 生成信件或文档 | 一个预订或预约应用程序 | 任何形式的用户界面 |
|---------|-------------|-----------|

|               |           |              |
|---------------|-----------|--------------|
| PDF 或 Word 输出 | 日历或日程安排集成 | 网页前端         |
| 真实的电子邮件发送     | 数据库服务器    | 任何部署在任何位置的内容 |
| 模板化的客户文案      | 登录或用户账户   | 移动端应用程序      |

现在看看记录中缺失的部分：没有信件内容，没有问候语，没有收件人地址，没有政策摘要，没有模板。没有任何东西被发送到任何地方。该记录就是你的代理输出——它是你的系统生成的东西，也是那个被标记的东西。

名称代表业务操作；代码是一条日志记录。issue\_decision\_letter 并不会编写一封信件；book\_slot 并不会预订任何东西；record\_decision 只是写入一行记录。它们的命名代表了其所代表的含义，因此治理论证便始终清晰可读——任何编写文件的工具都会使系统成为一个执行者，无论它写入哪个文件。而电子邮件程序、预订界面或文档生成器则属于不同的职责范畴，构建其中任何一个，都不会推动评分标准 1 中任何一个指标的进展。

标记实际所在的位置：智能体做出了哪个决策，在哪些证据基础上，经过了哪些判断节点，付出何种成本——以及如何将这些信息关联起来，并在你的评估数据集中证明这一点。一个团队若能展示出正确的决策及其可追溯的证据链条，其得分将高于一个为错误主张生成漂亮文书的团队。

团队容易过度构建的地方。当输出应为“请求缺失信息”时，只需要一个简短的字符串来说明缺失项的内容——例如“request: valid pre-authorisation for procedure 47120”。无需为客户准备的正式语体、语气或模板。

评测标准 1 中没有任何一项针对此字符串的表述。

任何工具都不能执行现实世界中不可逆的操作——不能发送真实邮件，不能实际付款，不能真实预订，也不能向任何在线系统写入数据。这是硬性规定，非权宜之计。Class 4 的笔记本正是基于这一原因才模拟了单次写入操作，即便该项写入最终落于文件中，该安全防护原则也绝不改变。

## D2 · 工具层 — 决定该智能体是否优秀的部分

三个部分。第 4 课将工具层称为 ACI（智能体-计算机接口），并指出：工具的描述和签名就是模型能够获得的全部手册。它无法询问同事、悬停查看提示、阅读你的源代码或在预发布环境测试。D2 中的所有内容皆由此而来。

### D2(a) · 工具集 — 应被精心挑选，而非简单累积

在代码库中的表格里，根据第 4 课提出的三个问题，对每个部署的工具进行评分：

| 问题                    | 一个好答案应该是什么样子                                                                                    |
|-----------------------|-------------------------------------------------------------------------------------------------|
| 1 · 如果没有它，某个任务真的会失败吗？ | 说出那个会失败的任务。从最小必要集合开始，只添加实际观察到的失败情况，而不添加任何想象中的失败。                                                |
| 2 · 模型是否会将其与邻近的情况混淆？  | 关键在于区分能力，而非数量。十个明显不同的工具没问题；但如果有三个工具功能重叠，就会带来麻烦。如果你无法用一句话说清楚何时使用 A 而非 B，那么模型也无法作出判断。             |
| 3 · 当它从未被调用时，会产生什么开销？ | 它的定义存在于提示前缀中，即每次交互开始时发送的固定文本块。无论是否被调用，该部分都会每天重新发送并重新计费。这增加了未被完全测试的错误暴露面。如果它产生了输出，就增加了一个额外的决策环节。 |

工作示例 — 四年级自己的五个工具，由笔记本自行评分。我们选择四年级的例子，而不是 A2 问题中的任何一个，以免任何团队直接得到自己的答案。请复制格式，而非内容。

| 工具             | 没有它会失败吗？               | 容易混淆吗？ | 为何它能占据一席之地           |
|----------------|------------------------|--------|----------------------|
| lookup_order   | 是的——没有任何其他内容能解析订单 ID   | 否      | 入口；其余所有内容都需要其输出      |
| check_shipment | 是——真正原因的唯一来源           | 否      | 唯一能反驳一个看似合理故事的工具     |
| 检查库存           | 是的，对于某个日期，库存可能会使承诺变得错误 | 否      | 便宜，而且能防止做出自信满满却错误的承诺 |

| 工具                  | 没有它会失败吗？         | 易混淆吗？    | 为何它能占据一席之地                                               |
|---------------------|------------------|----------|----------------------------------------------------------|
| search_notes        | 不                | 是的，有两个以上 | 故意留下作为陷阱。它失败了问题 1 和 2 — 而恰恰是这个工具造成了 5 节的失败。在生产环境中，你应该舍弃它 |
| send_customer_email | 是的 — 任务说的是“告诉他们” | 否        | 唯一的写入。一个网关覆盖整个代理                                         |

导致工具在问题 1 和问题 2 上失败的同样也是破坏代理的工具，这并非巧合——这就是论点。用你自己的数据集运行同样的三个问题，看看哪个工具看起来像 search\_notes。

在添加工具之前，尝试先不添加。按优先级顺序有四个步骤——在报告中说明你尝试了哪些。

1. 扩展现有工具的参数，而不是添加一个并列的工具。
2. 从一次调用中返回更多信息，而不是添加第二次查询。
3. 将步骤移出循环，放入代理运行前或运行后的普通代码中。
4. 然后才添加工具。

我们认可最简明可行的清单。四个有充分理由的工具胜过十一个。当你移除某个工具并指出导致其被移除的观察结果时，将获得明确的认可——这是任何人都不会自然产生的行为。

D2(b) · 工具描述符 —— 编写手册，因为它独一无二

每个工具都附带一个描述符规范。包含六个字段，无一例外：

|         |                                                                                                         |
|---------|---------------------------------------------------------------------------------------------------------|
| 名称 + 签名 | check_coverage(policy_id: str, procedure_code: str, site: Literal["SIN","KUL","HKG"]) -> CoverageResult |
| WHAT    | 一句话：这是唯一能回答的问题。                                                                                         |
| 输入      | 每个参数、其类型以及无效值的作用。                                                                                       |
| 退货      | 形状和一个大小限制。“最多 3 个记录，每个 40 个标记。”                                                                         |
| 失败情况    | 导致其返回空值或出错的命名条件。                                                                                        |
| 不可逆？    | 是 / 否。如果是，请说出覆盖它的门的名称。                                                                                  |

然后应用至少两种防错措施（即设计上的改动，使某类错误根本不可能发生，而不是要求模型小心）。请参考第 4 类的表格：

| 之前           | 之后                                 | 它使得以下情况成为不可能     |
|--------------|------------------------------------|------------------|
| site: str    | site: Literal["SIN-DC1","KUL-DC2"] | 拼写错误的站点静默返回“无记录” |
| 患者姓名         | 患者 ID                              | 错误的涂抹            |
| 发送邮件(...)    | draft_email(...) + send_email(...) | 发送时误以为是草稿        |
| dry_run: 布尔值 | dry_run: bool = True               | 不可逆的默认设置         |

对于你们的两个工具，分别说明每个工具使得什么变得不可能——不是什么被劝阻了。

并实施一次重构。选择一个工具，发布其描述符和返回结构的 v1 和 v2 版本，并记录每次调用返回的 token 数量、评估通过率以及分别通过的护栏案例数量。这是将第 4 章第 7 节的原则应用到你们自己的工具上。其所强调的重点，也是我们希望标记的重点在于：提示指令的费用在每次运行的每次调用中被持续收取，而当你更换模型时便会失效；而接口约束只需支付一次且持久有效。如果你们的 v2 版本并非更小或更安全，请明确指出——一个未带来帮助但被诚实报告的重构，其评分优于从未被衡量过的重构。

D2(c) · 一轮中调用多个工具——成本杠杆

第四类循环每轮仅解析一个 Action。然而，实际的工具调用 API（应用程序编程接口）并非如此工作：模型可以在一次响应中返回多个工具调用，而一个设计良好的循环会一起执行这些调用，并在再次询问前合并所有的观察结果。

这一点很重要，因为回合是昂贵的维度。根据第 4 类胶囊 3 派生而来，以及第 5 类将其编码化：循环是无状态的，因此每回合都需要重新发送整个轨迹。对于  $T$  个回合，一个基数提示  $B$  和每个回合增加  $D$  个标记：

|                                                 |                   |
|-------------------------------------------------|-------------------|
| 输入 $\sim B \times T + D \times T^2/2$           | 第 4 类胶囊 3，幻灯片上的形式 |
| $\text{input} = B \times T + D \times T(T-1)/2$ | 第五类的笔记本，精确的和      |

两者模型相同——第一个忽略了  $-1$ ，因为这不影响论点。当  $T = 12$ ， $B = 1,200$ ， $D = 400$  时，前者结果为 43,200，后者为 40,800。两者皆可使用，请说明你选择哪一个。前缀  $B$  每一回合重新发送，累积的历史信息随回合数  $T$  呈二次增长。减少回合数会同时减少两项结果，第二项减少明显，且“回合数从八增加到十六，并不会使费用翻倍，而是几乎变为三倍”。

A2 所需求的内容

- 1 · 扩展循环使其能够在单轮中解析并执行一组工具调用。具体来说：解析一个 Action: 块而非单行的 Action:，对每个调用进行执行，并追加每个观察结果。提供的起始模板（Monday）已包含此功能。
- 2 · 记录依赖规则——明确哪些工具可以一同调用，哪些不行。仅当两个工具不相互依赖对方的输出时，该组合才可并行执行。
- 3 · 进行衡量。在相同的评估集上分别进行串行和并行运行，报告两种情况下的轮次、标记数和成本。
- 4 · 展示正确性没有发生变化。保持相同的通过率，或解释其中存在的差异。

示例工作 — 问题 A，以及为何这样做值得。第 1 轮必须单独运行：get\_claim(claim\_id) — 其余所有操作都需要该函数返回的会员、医院和明细项信息。但一旦得到数据，每条明细项的校验就仅依赖于该记录本身，而策略和医院信息查询亦不依赖其他内容。因此，它们可以一起执行：

|        |                                                                         |              |
|--------|-------------------------------------------------------------------------|--------------|
| 第 1 轮： | get_claim("CLM-8842")                                                   | -> 返回 3 条明细项 |
| 第二回合：  | lookup_policy(成员编号)                                                     |              |
|        | check_coverage(47120)    check_coverage(31255)    check_coverage(62480) |              |
|        | get_hospital_status(医院编号)                                               |              |
| 第三轮：   | get_preauthorisation(成员 ID, 62480)                                      | 仅此一行需要       |
| 回合 4：  | 下达决定函(...)                                                              | ← 在闸门之后      |

注意并行处理能合并什么，又不能合并什么。由于三次覆盖率检查彼此独立，因此它们可以折叠为一个回合。而预授权则不行——你不知道哪一行需要预授权，直到覆盖率检查得到回复。这就是你的依赖关系规则，任何无需预授权的索赔会提前一整个回合完成。

六个连续回合减少为三个。当  $B = 1,200$  且观测总量相同时：

|    | 转换              | 输入标记 $\approx B \times T + D \times T(T-1)/2$ |
|----|-----------------|-----------------------------------------------|
| 顺序 | 6 ( $D = 400$ ) | $7,200 + 6,000 = 13,200$                      |
| 并行 | 3 ( $D = 800$ ) | $3,600 + 2,400 = 6,000$                       |

同样的工作，同样的观察结果，输入的词元减少了 55%——因为历史记录只重复发送了三次，而不是六次。没有任何内容被裁剪；只是对调用进行了重组。

有两个诚实的限制，我们期望你能找到它们。并行调用可能会增加成本，因为某个调用被发现是不必要的——而在串行情况下，早期的观察结果本可以提示你跳过它。同时，它们移除了模型原本可以用来做决策的一个关键点。请说明你的依赖规则在何处划下界限以及原因。

D3 · 安全护栏层以及安全护栏检查清单

两件事情，而且它们是不同的。

- (a) 代码层，在任何提示微调之前部署：一个步骤上限、一个预算上限、动作去重，以及一个明确的自主性设置——即建议、确认或采取行动，并将关卡设置在不可逆步骤之前，而不是在整个智能体之前。请说明你选择的设置，并在报告中进行辩护。
- (b) 守门员检查清单——至少 10 个测试用例。这些与评估用例不同。评估用例关注的是它是否正确完成了任务，而守门员用例关注的是在应当拒绝、限制或升级的情况下它是否确实这样做了？这十个用例中的每一个都必须说明其旨在捕捉的错误行为，并陈述实际观察到的结果。第 6 类（负责任的人工智能、失效模式和前沿问题）将

提供 OWASP（开放全球应用安全项目）2025 年 LLM（大型语言模型）十大风险作为类别来源——但你需要的代码层内容在第四节课中已有，因此不必等到第六节课才开始。

你的十个案例中至少有三个必须涵盖请求文本本身具有恶意性质——这两个问题中都包含由组织外某人编写的自由文本：成员的索赔叙述内容和转诊医生的临床摘要。

D4 · 评估数据集——30 至 50 个案例

您自己问题的相关案例。

- 共 30 – 50 个案例，由团队撰写。如果是 6 或 7 人团队，则每人撰写 5 – 8 个。
- 按结果评级 —— 评分标准是代理得出的结论，而不是它达到该结论的过程。
- 负面案例 —— 最少 2 个，但本规模案例集应有 6 到 10 个。一个负面案例是指正确结果应为拒绝（refuse）、询问（ask）或升级（escalate）——而不是执行某个操作的真实案例。每个负面案例必须明确指出它旨在捕获的是何种错误行为。那些在你们开发过程中真正触发并导致改动的负面案例会获得额外加分。
- 每个案例进行多次试验（3 次是一个合理的默认值），因为同一个案例并不总是产生相同的答案。
- 隔离 —— 每个案例都从一个干净的状态开始；没有任何一个案例可以依赖于前一个案例的执行结果。
- 混合评分机制 —— 在自动检查可以做到诚实的情况下使用自动检查，而在无法保证诚实性时使用人工评判（如由 LLM 作为评判者或指定的真人）。仅仅因为筛选子串正确匹配但理由错误，是一种真实存在的失败模式，Class 4 已经向你展示过这种情况。
- 按照策略和模型分别报告经过测量的通过率，并注明试验次数。

为什么是 30 到 50 而不是 10。10 个案例的集合无法区分两种策略。第四类的演示集合之所以能区分出一个谨慎的智能体和一个看似合理的智能体，正是因为其中包含了负例——而且 8 个案例中只有 6 个是普通的。正是规模使得这个数字有了意义。

D5 · 模型电池——一次脚本运行，三个实时模型

两次相同的接口运行，而这一部分使得接口成为一种工具，而非一种形式。

(a) 脚本化运行。您提交的接口必须在脚本后端上完成端到端运行——使用确定性的预设响应，无网络，无密钥——以便评分者可以克隆仓库并零成本复现您的结果。BACKEND =

“scripted” 在提交的代码中必须为默认设置。如果代码不能以这种方式运行，则技术执行将受到限制。

(b) 活体电池测试。通过 OpenRouter 对至少三个不同的实时模型运行相同的评估集，并报告其相对性能——不是抽象地判断哪个模型“最好”，而是要明确在具体成本下哪个模型能够胜任你的工作，并找出它们性能的差异点。预期它们在负例上的差异会最大；这正是你需要关注的重点发现。

你的代码必须与供应商无关：顶部仅包含一个 BACKEND / MODEL / BASE\_URL 模块，且只有一个函数知晓供应商的存在。切换模型仅需修改一个字符串。这属于 Class 4 模式；请直接照搬此模式。

根据预算来确定电池尺寸——参见第 7 节。完整的评估集 × 1 次试验 × 3 个模型，再加上仅针对负样本的 3 次试验，是我们所预期的方案。将模型分配给团队成员——每个成员负责一个模型，并在他们自己的密钥上运行。如果有 6 或 7 名成员，则如果你想的话，还有空间再增加第四个模型。

D6 · 服务成本模型——三层架构及你操作的四个杠杆

使用 Class 5 的三层模型。你已在 Class 5 的服务成本笔记本中构建了此模型；请复用该笔记本，而不是重新开始。使用你在 D4 中测得的成功率以及在 D5 中测得的标记（token）数量——不要使用估计值。

| 层            | 什么是它                                | 随着规模的扩大                          |
|--------------|-------------------------------------|----------------------------------|
| 1 · 按任务计费    | 输入 token 数 + 输出 token 数 + 任何检索或工具费用 | 线性的。从不摊销。                        |
| 2 · 每任务的预期回退 | (1 - 成功率) × 处理一次失败的成本。              | 线性的。从不摊销。通常最大层，而几乎所有成本模型都会忽略这一点。 |
| 3 · 固定月费     | 存储、基础设施、您的评估运行、监控和维护。               | 每项任务的成本将永远降低。                    |

|           |                                 |                               |
|-----------|---------------------------------|-------------------------------|
| 输入标记      | ~ B*T + D*T^2/2                 | B = 基础前缀, D = 增长/回合数, T = 回合数 |
| 第一层变量     | = 输入*价格_入 + 输出*价格_出 + 检索 + 工具费用 |                               |
| 第二层备用方案   | = (1 - 成功率) * 失败成本              |                               |
| 每项成功任务的成本 | = 第 1 层 + 第 2 层                 |                               |
| 每月        | = 每项成功任务的成本 * 调用量 + 第 3 层       |                               |

请严格基于此构建你的基准。使用模型标价计算的原始输入和输出标记数量。不要在基准中引入任何更复杂的内容——下面的两项调整确实是真实的，但它们依赖于具体模型，会使你的核心指标变得难以验证。

成本层 2 基于人工成本，而非猜测得出。即使设计中未指明任何具体人员，失败也不是免费的。

失败成本 = 小时工资率 × 每次升级所需分钟数 ÷ 60。附录 A 中的每个问题陈述均提供了默认角色及处理时间，可直接采用，或替换为自己的数据并说明理由。

两个因素会变动该数值，而两者都不应包含在基线中。

请先报告您的基线数值。若上述两个因素中有任何一项适用，请在旁边报告调整后的数值，并说明发生了什么变化。

(a) 提示词缓存会拉低账单——有时降幅相当可观。

你的前缀 B 在每轮中都是相同的文本，而这恰好是缓存存在的目的：提供商将其存储起来，并仅收取一小部分输入价格来重新读取它。在按二次方计算的情况下，这确实是一种真正的节省，因为 B×T 是缓存所影响的价格项。

然而，该机制、折扣方式和计费模式因服务商和模型的不同而有所区别，并且它们还可能发生变化。有些服务商会自动缓存；有些则要求你手动标记某段内容；有些则需要你预先付费写入缓存，才允许你在读取时节省费用；还有一些缓存会在几分钟后过期。此外，还存在一个可缓存内容的最小尺寸，而且缓存命中条件是前缀完全不变——如果你的系统提示词开头添加了时间戳，不知不觉中就已经关闭了缓存功能。

因此，请勿假设缓存效果。实际测量它。如果你使用了缓存，请用同样的评估数据集测试有和无缓存的情况，采用 API 返回的分词数量而非计费表中的估算值，并同时报告两种情况的数据。一个称“我们实测该模型的输入账单因缓存降低了 X%”的团队，比起一个单纯假设折扣的团队，显然工作做得更扎实。

(b) 一个推理模型会使账单上涨，而你很可能没有注意到。

推理模型在回答之前会产生隐藏的思考过程。这些思考的标记会被计为输出标记，而输出是收费较高的部分：在第 7 节的低价层级，输出成本是输入价格的 4 倍；在前沿层级则为 5 倍。一个默默产生 900 个思考标记的模型，而普通模型只输出 100 个，并没有让你的代理略贵一点；它实际上已经改变了你的账单结构。

如何发现：查看 API 返回的使用量区块，而不是你自己的估算。推理标记会出现在输出计数中。如果每轮的输出量跳增了一个数量级，那么就是发生了这种情况。

如何通过 OpenRouter 限制它：请求中添加一个推理对象（reasoning object），而具体使用哪个字段，则取决于模型的家族。

|                                   |                                       |
|-----------------------------------|---------------------------------------|
| "reasoning": {"max_tokens": 1024} | // Anthropic、Gemini 和部分 Qwen - 存在硬性上限 |
| "reasoning": {"effort": "low"}    | // OpenAI 的 o 系列和 GPT-5、Grok - ~20%   |
| "reasoning": {"enabled": false}   | // 关闭状态时，模型会允许此功能                     |

努力程度 以比例而非绝对值表示——大致为最低 10%，较低 20%，中等 50%，较高 80%。Anthropic max\_tokens 下限为 1,024。

陷阱，它会困住人们

reasoning: exclude = true 可阻止将推理结果返回给你。模型仍然会执行推理，你也仍然需要为此付费。排除只是整洁性设置，并非成本控制措施。如果你想节省开支，请设置上限或关闭该功能。

我们对 A2 的建议是：不要使用。这个作业奖励的不是推理模型——分值在于系统集成，而推理产生的 Tokens 很快就会耗光一个价值 10 美元的 apikey。如果你一定要用，请设定限制，说明你设了什么限制，并分别报告启用水准后和关闭水准后的成本。这种对比本身就是一个有价值的发现。

使用敏感性分析表，而非单一估计值。你的成功率是一个估计值，失败成本也是如此。应展示在成功率上下各 ±10 个百分点的范围内，每次成功任务的成本，并说明你的结论在这一整个区间内是否依然成立。一个稳健的答案

以及尖锐的答案需要不同程度的信心，赞助商有权知道你交给他们的是哪一种。

盈亏平衡成功率——这实际上决定了你要部署哪个模型。

你在 D5 中的模型电池为你提供了每个模型的实测成功率。成本较低的模型每次运行的费用更少，且能获得更多的错了；每个错误答案都会让你付出升级的代价。因此，真正的问题不是“哪个模型更好”——而是：

便宜模型的性能需要好到什么程度，才能让它每次节省的金钱不至于被其引发的失败所吞噬？

三个你已经掌握的量：

|   | 什么是它                                    | 它来自哪里             |
|---|-----------------------------------------|-------------------|
| C | 每次在低价模型上运行的成本——仅包含 Token 数量             | 您的 Token 数量 × 单价  |
| E | 每次在高价模型上成功任务的成本——包含其 Token 数量以及其自身的失败成本 | 该模型的第 1 层 + 第 2 层 |
| F | 一次故障所付出的代价——升级的必要性                      | 你问题的图表见附录 A       |

一行逻辑：切换为廉价模型后，每个任务可节省  $E - C$ 。一次故障的成本是  $F$ 。因此，在节省的费用被耗尽之前，你能承受的故障次数为  $(E - C) \div F$ ，因此你需要的准确率就是剩余的部分：

|          |   |                   |
|----------|---|-------------------|
| 你能承受的故障数 | = | $(E - C) / F$     |
| 盈亏平衡成功率  | = | $1 - (E - C) / F$ |

为什么  $E$  包含了它的失败，而  $C$  没有。对于昂贵的模型，你测量了它的成功概率，因此你可以估算出它的失败概率并将其计入。而对于廉价的模型，它的成功概率正是你要求解的未知数，因此它不能出现在求和那一侧。这种不对称性正是这个公式看起来不太对称的全部原因。

以问题 A 为例。采取一个廉价层级的运行，花费为 0.005 美元，以及一个中等层级的运行，花费为 0.049 美元。中等层级的模型在你的数据集上测量的成功率为 92%，而一次失败的索赔会使索赔评估员损失 7.60 美元。

|                                              |                    |
|----------------------------------------------|--------------------|
| $E = 0.049 + (1 - 0.92) \times 7.60 = 0.657$ | 所有费用包含累积           |
| $E - C = 0.657 - 0.005 = 0.652$              | 通过节省成本，在每个任务上都能省钱  |
| $0.652 / 7.60 = 0.086$                       | 因此你能承受 ~ 8.6% 的失败率 |
| 盈亏平衡点 $p = 1 - 0.086 = 0.914$                | 便宜的模型需要达到 91.4%    |

现在读一下这才是练习的重点。便宜的模型每次运行的成本大约只有贵的模型的十分之一——但它仍然必须与昂贵的模型结果相差不超过一个百分点才能证明是值得选择的。当失败成本很高时，模型的价格几乎无关紧要，重要的是准确性。

反之亦然。如果失败的成本从 7.60 美元降到 0.76 美元，同样的计算将得出盈亏平衡点为 86.2%——这意味着便宜的模型在可以承受约 14% 的错误，而不是 9%。出错的成本越低，模型价格的重要性就越大。这正是附录 A 为每个问题给出不同失败成本的原因，也是为什么同一个代理在某个问题上 是好的决策而在另一个问题上则可能是坏决策的原因。

然后为自己的系统回答这个问题：你的便宜模型是否达到了它的盈亏平衡点，或者还差多远？用一句话说明，将你的两个实测数据合并其中，这是报告中最实用的内容。

为什么这不是第四课第三胶囊中的“÷ 成功率”公式？这两种方式都是正确模型，但对应不同的世界。用成功率去除意味着失败时需重试直到成功——一次失败就会让你多一次尝试。而加上  $(1-p) \times \text{失败成本}$  这种方式，则是在失败时代价会升级——一次失败会给你带来一个真人。在这两个 A2 问题中，错误的结果都会传递给一个人，而不会返回到循环中，因此升级形式才是正确的。用一句话说明你的系统处于哪个世界；使用重试公式并为此辩护的团队并未犯错。

成本账本——这是我们要特别标注的部分。一个只报一个数字的成本模型，无法告诉我们你的设计情况。请列出你操作的四个杠杆，并为每个杠杆提供调整前后的实测数据。

| 杠杆         | 它攻击的对象                                | 你在哪里构建它           | 报告                      |
|------------|---------------------------------------|-------------------|-------------------------|
| 1. 工具块大小   | B — 每回合均重新发送，无论是否被调用。与回合数成线性关系。       | D2 (a) — 你用来切割的工具 | 在您进行裁剪之前和之后的工具定义标记      |
| 2 · 转数 T   | 二次项。你拥有的最大的杠杆。                        | D2(c) — 并行调用      | 回合与输入标记，顺序与并行处理         |
| 3 · 观测规模 D | 复合项 — 每一次观测在后续每个回合中都会被重新发送            | D2(b) — 描述符重写     | 每次调用返回的标记数，v1 与 v2      |
| 4 · 成功概率   | 设置第二层，通常是最大的一层。一个错误率高达一半的廉价代理实际上并不便宜。 | D4 — 评估数据集        | 测量通过率，以及由此推算出的每次成功任务的成本 |

杠杆 1 和杠杆 3 看起来相似，但实际上不同：一个“粗的工具块”在操作中是线性增长的，而一个“粗的观察结果”会引发复合效应。两者都值得削减；但只有其中一个会导致数量级的暴涨。请说明是哪一个造成了账单的主要部分，并解释你如何得出这一结论。

然后指出随它提供的三个限制条件：单步限制、预算上限、每个用户的月度限额。

这标记的是方法，而不是算术精度。一个具有明确假设、三个分离层级、敏感性范围和四个测量杠杆的可辩护模型，胜过一个没有上述任何要素的自信单一数值。

### D7 · 重现两个失败案例——其中一个为循环失败

重现两种不同的故障模式，并在它们所属的层级中分别修复——代码、工具接口或提示。每一种修复都必须作为你当前工作代理的删减版本实现，而不能是单独编写的错误代理：“工作代理，减去 X。”

将 X 恢复后必须能重现原有行为。

失败 1 · 一个循环控制失败——这个是必需的

代理在不断地循环：重复它之前已经执行过的动作，或者重新读取它已经知道的信息，或者永远无法得出结论。Class 4 正是构建了这种情况——一个删除了某个守卫的仔细的守卫链——其结果正是值得理解的地方：

8 轮之后，无任何答案，成本增加了 1.6 倍，且没有任何异常被抛出。它没有崩溃。没有任何东西失败。它只是在原地浪费资源——而循环本身如果没有额外的机制，是不记录它自己行为的。你只有在计数时才能发现这种情况。

需要报告的四个要点：

- 1 · 用于发现该问题的监控工具。每次运行的轮次和成本记录。如果你没有察觉到问题，就无法报告这个问题。
- 2 · 整个评估集中的轮次分布情况——中位数、最坏情况以及有多少次运行达到了你的步骤上限。

单个数值并不能代表一个分布。

- 3 · 在代码层面的修复方案——去重操作、步骤上限、预算上限。说明哪一个真正解决了问题，并解释另外两个为何无法解决问题。
- 4 · 开始前与开始后：回合数、标记数、成本与通过率。能阻止失控的步骤上限，也同样会截断正常的长流程——请证明你的通过率并未下降。

根据证据来设定上限，而非凭一个整数。如果你的中位运行回合数是 4，而最糟糕的合法流程是 7，则设定 8 的上限是合理的，设定 30 则纯属装饰。请说明你上限的设定依据。并且要让停止效果清晰可见。一个静默返回空答案的上限，比循环问题更糟——它把一个可见的成本问题，转变为一个隐形的准确性问题。

失败 2 · 不同的层面

你可以自由选择，但这必须在工具界面或提示中体现，而不能再次运用在循环控制中。第四类的情况示例是一个重复、陈旧且暗藏雷区的观察结果——246 个标记，最终产生一个自信但错误的答案，最后是在界面层面修正，而不是在提示中增加一句话即可解决的。

对于两种故障，说明修复措施属于哪一层，并解释为什么另外两层是错误的选择。这种判断是评分的主要部分。

4 · 你需提交的内容

四个成果物，打包在一个名为 PE6201\_A2\_[团队 ID].zip 的压缩文件中——即课程开始时分配给你的团队标识符，例如 PE6201\_A2\_B-4.zip。同时包含视频链接。

| # | 成果物                | 说明                                                                                                                                                                                                                                                                     |
|---|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | 代码仓库 — 两处存放        | (a) 团队在 GitHub 或类似平台创建的公开代码仓库；(b) 相同代码文件的副本存放在 NTULearn 提交文件夹内。两者均需提供：仓库可展示项目历史和每位成员的贡献，而文件夹内的副本则确保评分不受链接能否正常工作的影响。请包含一个 README 文件，指导不熟悉项目的陌生人从克隆代码开始，直至成功运行出可复现的脚本流程。仓库中应包含 AI 代理、工具、护栏层（guardrail layer）、评估数据集、测试框架（harness）、测试用例（fixtures）、结果表格及成本模型。所有数据和数值均存放于此处。 |
| 2 | 团队报告 — 不超过 2,000 字 | 论点，而非数据。其结构如下，且必须以“为何需要一个代理”开场。表格和图表不计入字数限制；正文内容则计入。                                                                                                                                                                                                                   |
| 3 | 录制的演示 — 5 分钟       | 系统运行演示，现场展示一个失败案例，并附上相关数据。每位成员均需发言。超时将在“沟通”项下被扣分。                                                                                                                                                                                                                      |
| 4 | 团队自评               | 每队一份集体表格——不是每人一份。大家共同完成，根据评分标准 1 对团队进行评分，并附上一段关于你们权衡取舍的简短反思。与所有其他材料一起提交到你的 A2 提交文件夹中。此项不评分，但必须提交——缺少自评将被视为未完成的提交。                                                                                                                                                      |

贡献记录。仓库内必须包含一个名为 CONTRIBUTIONS.md 的文件，明确记录每个人完成了哪些工作，且提交历史需能佐证该记录。第 8 节的评估依赖于此。

报告——需遵循规定的结构，并注意字数限制

六个部分，按此顺序。各部分预算仅供参考；总字数 2,000 字为上限。数字、表格和图示存在于仓库中，不计入字数。

| # | 章节      | 里面放什么                                                                           | 文字  |
|---|---------|---------------------------------------------------------------------------------|-----|
| 1 | 为什么是代理人 | D0 — 当前所处的层级以及层级 1 到 6 分别能与不能实现的内容；两个测试，包括你自己的 $s = P^{(1/T)}$ 计算结果；良好的样子是什么样的。 | 400 |
| 2 | 工具层     | D2 — 选择这一组的原因及被舍弃的部分；你描述符重写的衡量结果；你的依赖规则以及并行调用带来节省的部分。                           | 450 |
| 3 | 证据显示了什么 | D4 + D5 — 通过率，即模型出现分歧的情况以及未能识别的负样本。                                             | 350 |
| 4 | 它要花多少钱  | D6 — 四个杠杆的作用及其主导性，你问题的月度规模、敏感性范围和盈亏平衡点。如果适用，请在此处注明缓存或推理模型。                      | 400 |
| 5 | 两个故障    | D7 — 循环故障，你是如何发现它的，修复方法以及修复前后的对比；第二个故障及其对应的层级原因。                                | 250 |
| 6 | 我们不会部署  | 你所发现的限制，外加一段关于你未曾构建的架构的说明——它本可以发现什么，本会付出什么代价，为什么你坚持使用单智能体。                      | 150 |

预算的分配取决于分数所在的位置，以及哪些论点难以简明扼要地阐述。第 1 和第 2 部分一共 850 词——将近一半——因为概念理解（25%）和推理与论证（25%）占了评分标准的一半，而几乎所有这些内容都集中在使用这个层级和这组工具的原因上。第 4 部分有所扩展，因为现在的成本模型有四个可量化的控制杠杆和一个需要捍卫的范围，而不再是陈述一个数字。第 5 部分特意写得较短：两个失败案例，描述得紧凑，比两个冗长叙述更有说服力。

第一部分是最先阅读且最难的部分。一份以团队构建了什么作为开头的报告，而非说明为何代理是合适的工具，已经跳过了整个第四课所围绕的核心问题。

5 · A2 的评分方式

量规 1，共四项标准，适用于团队提交。各项标准在 A2 项中的具体要求如下：

| 评分标准          | 此处获得最高评分的关键是什么                                                                                                                                                                                                                            |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 概念理解 —— 25%   | 你可以将问题放在梯子上防守梯级——多么单一的提示，固定的工作流或只读代理检索所能实现和无法实现的内容，以及其所付出的成本。预先构建的诊断工具指明了一个可以以机器速度反驳模型的权威系统。你可以明确指出 ReAct 循环在何处不再是最合适的架构，并提供证据而非凭主观判断。评估用例与护栏用例之间的区别始终得到维持。自治设置的选择是基于是否涉及不可逆的步骤，而非主观偏好。                                                   |
| 技术执行 — 30%    | 该脚本化运行会重现。每次运行都会记录回合数和成本，因此循环失败是你实际检测到的结果，而非假设。门控动作会记录一个决策及其证据链和门限条件——在文档、接口或集成上花费的精力在此处毫无意义。该循环在同一回合内确实会执行多个工具调用。每个工具都附带一个包含完整六项描述符的定义，并且在签名中包含了真实的事前防错机制。评估集是孤立的、结果分级的、多轮次的，并包含真实的负样本案例。防护措施由代码实现，而非通过提示引入。实验的实时电池运行成功，所有数据均为代码实际生成的结果。 |
| 推理与论据 — 25%   | 一个关于你并未构建的架构的论述段落——说明它本可以发现什么，会付出什么成本，以及为何你坚持使用单智能体——并附有证据支撑。该工具清单是能够经受辩驳的最简列表，根据三个问题进行评分，其中至少移除了一项工具，或记录了“我们曾尝试但未添加”的情况。描述符的重构是经过权衡的，并且即使未能提供帮助也会如实报告。成本账本列出了所有四个关键因素，并指出哪一个占主导地位。模型选择依据的是你自己的证据。两项失败的定位准确，明确指出了出错的层级，并指出了用错的其他层级。       |
| 沟通与清晰性 —— 20% | 报告以论证为主，而非叙述。演示不仅展示了成功路径，也展示了失败的情况。代码库的结构清晰，即使是团队以外的人也可以顺利导航。限制条件被坦率地公开声明。                                                                                                                                                                |

工作的重点所在。在技术执行与推理方面，架构框架、安全防护机制和证据积累是主要得分点。一个虽然呈现了更多功能但证据不足的智能体，得分会低于一个功能较少但能提供充分证明的智能体。

6 · 利用 AI 构建 AI

鼓励实施，条件与 A1 相同，四点不变：

- 1 · 你必须能够解释所提交的每一段代码——解释它做什么以及为什么放置在那里。按代码块而非代码行来评估：你不是在被测试语法知识。A2 适用于整个团队：任何成员都可能被问到任何代码块的相关问题。
- 2 · 报告中的图表数据应是你自己运行得到的测量结果，而非模型为你生成的数值。
- 3 · 所使用的资料 and 工具需注明来源。
- 4 · 不得重复提交任何内容。A2 作业不得出现在任何人的课程结业项目中， likewise，任何项目工作也不得出现在 A2（第 1 部分）中。

7 · 预算——10 美元密钥

每位成员持有一个个人 OpenRouter API 密钥，整个课程期间总费用不超过 10 美元，且中途不追加预算。A2 阶段的实时运行预算必须在扣除 A1 阶段费用后的余额范围内，并考虑课程结业项目的要求。

单次执行 ~12 约产生 43,200 个输入标记（tokens）和 1,200 个输出标记——在 Class 5 价格表上对应 Class 5 的计算量。价格单位为每百万标记的美元，数据截至 2026 年 8 月 28 日，与 Class 5 成本计算笔记中的数值相同，因此你们可以复用那份笔记来重新评估你们自己的运行预算。

| 等级 | 价格（内外部）      | 单次运行       | 40 例 × 1 次试验 | + 8 个负面项 × 3 次试验 | 整个电池，一个成员  |
|----|--------------|------------|--------------|------------------|------------|
| 便宜 | 0.10 / 0.40  | ≈ 0.005 美元 | ≈ 0.19 美元    | ≈ 0.12 美元        | ≈ 0.31 美元  |
| 中期 | 1.00 / 5.00  | 约 0.049 美元 | 约 1.97 美元    | 约 1.18 美元        | 约 3.15 美元  |
| 前沿 | 5.00 / 25.00 | 约 0.246 美元 | 约 9.84 美元    | ≈ 5.90 美元        | ≈ 15.74 美元 |

再看最后一行。在前沿模型上运行一个完整的测试集，其费用超过了课程提供的全部预算。这并不是我们设下的陷阱——这正是从第 3 周以来课程一直在引导大家做出的决策。因此，“低成本套餐”才是评估测试集的正确选择。如果你想在对比中使用前沿模型，请仅在错误案例上运行，并在报告中明确说明你所采取的方式。

三条让你不超预算的规则。

- 在脚本化后端进行调试。实时接口上的令牌仅用于最终的测试集运行，而不是为了查找你的错误。
- 每个成员负责一个模型。三个模型，三位成员，三把钥匙——而你的团队有 6 到 7 把钥匙，因此没有人需要独自承担全部电量。
- 如果你估算的每位成员实际开销超过 3 美元，说明你的电量预估过大——应减少试验或案例数量，或将某个模型降级处理，并在报告中说明你做了上述调整以及原因。预算需在固定额度内进行，这正是本课程的教学内容之一；我们并不为此致歉。

电量耗尽是一种可诊断的情况。尽早发邮件，我们将帮你找出电量消耗去向并制定一个利用剩余电量完成任务的计划——而不是提供额外积分。

8 · 团队、贡献与搭便车条款

你的团队已经确定。A2 项目将在课程开始时为你分配的小组中进行——A-1 至 A-9，B-1 至 B-9，C-1 至 C-10。A2 项目的小组不进行重新调整、更换或合并。如果你不确定自己属于哪个小组，请在签到日前询问，而不是之后。不同课程单元之间的联合小组不存在，也无法创建——签到和同伴评价均按各个单元分别进行。

团队声明 — 截止日期：2026 年 9 月 4 日（星期五）23:59（新加坡时间）

非邮件提交。请将一个文件提交至你的 A2 提交文件夹，文件命名为 TEAM\_DECLARATION.md（或 .pdf），文件内容包括：

- 团队 ID 和单元信息——例如 B-4，单元 B
- 每位成员的姓名和学号。
- 你选择的问题——A 或 B。
- 任务分工——根据常见问题解答（FAQ）的内容，或你们自己划分的任务。将姓名与具体工作对应起来。
- 你的代码仓库（repository）URL。
- 贡献声明 — 这一项不是可选的。

关于第 6 点，清楚地说明

请说明以下两件事之一。要么声明“所有成员都在做出贡献”——请明确表达，不要留空。或者指出存在的问题：如果某位成员没有参与，没有回应，或未能履行之前商定的任务，请在这里指名说明具体情况，包括之前达成的何项协议以及现在发生了什么。

这件事的重要性超出表象所见。设置这个检查点，是为了在问题出现时仍有时间补救。例如，如果团队在 9 月 4 日指出某位成员未贡献，教师将发出书面警告，该成员有九天时间补救——通常足够，且大多数问题都能解决。反之，如果团队在 9 月 4 日选择沉默，等到 9 月 13 日才提出同样的问题，其理由就相当薄弱：因为此前没有发出过警告，没有提供补救的机会，也缺乏同时期的记录。检查点处的沉默将被视为“所有成员都在做出贡献”。

在这里指出问题是善意的举动，并不会因此扣分。它开启了一个有截止日期的讨论。

此项声明是强制性的。没有提交任何内容的团队，将被视为提交不完整。

评分。导师会根据评分标准 1，只对团队提交的内容进行一次评分。所有成员都将自动获得该团队评分。

同伴互评——将于 9 月 13 日开放，于 2026 年 9 月 16 日（星期三）23:59（新加坡时间）截止。提交后，每位成员需通过 NTU 的小组项目同伴互评功能为队友的贡献情况进行评分（自我评分除外）。此项评分不会影响任何人的成绩，仅为一个信号：如果某成员被标记为贡献度低或无贡献，导师将审查证据并仅调整该成员的成绩，而其他团队成员仍保持团队成绩不变。按时完成此项互评是参与课程的要求

要求。

不作贡献。经记录的不贡献者在 A2 中将获得 0 分，而非团队总分。此处分仅在获得两个独立信号佐证时才予以应用：多数队友通过同行评价工具标记该成员，并且在提交历史或贡献日志中无任何可追溯的工作记录，同时获得教师证实。仅有同伴意见不足以构成评分依据。若团队在 9 月 4 日的检查点提出该问题，则先发出书面警告并开启补救窗口。学生可以回应，且适用标准的单次成绩复审请求程序。

9 · 您将获得何种反馈，以及何时获得

简单来说，以免有人在等待根本不会到来的事情。反馈的具体时间由任课老师自行决定，因此在特定日期去催促并不是一个好做法。

| 在             | 你将获得                                                                                                                                         | 何时                |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| A1            | 您的评分以及通过 NTULearn 提供的书面评语。<br>评语将集中于评分标准中的某一项，并针对您的自我评价与评分之间存在显著差异的地方。                                                                       | 评分时间由教师提供<br>自行判断 |
| A1 — 按班级进行的分析 | 本学期暂不举行。关于 A1 成绩分布或常见错误的全班分析会议将不举办。A1 的截止日期与 A2 发布日期同天，而第 5 节课则是在第二天，因此不存在适当准备此类分析会议的时间。做出这一有意识的决定，是为了确保个别反馈能及时、完整地传达，而不是因为全班反馈会议准备得太晚而流于表面。 | —                 |
| 你的项目问题陈述      | 在模板的教师反馈框中填写书面反馈。                                                                                                                            | 时间安排在教师处<br>酌情处理  |
| A2（本次作业）      | 反馈将在提交后在线下分享——第 6 周为最终授课周，之后不会再有课程进行讨论。                                                                                                      | 时间安排指导教师的<br>酌情处理 |

对于 A2 来说，这意味着什么？请不要打算将 A1 的反馈作为你判断一个优秀的 A2 应该是什么样的依据。本简报的第 5 部分才是该依据——它逐项说明了此作业的最高等级应具备的特征。请在开始前阅读，而不是完成后才去阅读。

10 · 时间表 日期

|                                   | 什么                                                                                |
|-----------------------------------|-----------------------------------------------------------------------------------|
| 2026 年 8 月 30 日，星期日               | A2 已发布。请选择你的问题，然后在编写代码前先写下 D0——即阶梯、诊断、理想状态的模样。接着是 D1、D2 和 D3 的代码层。所有这些都是你已有的四级内容。 |
| 2026 年 8 月 31 日，星期一 / 9 月 1 日，星期二 | 第 3 课 — 服务成本与投资回报率（Return on Investment, ROI）。提交至 D6。请带上您问题的相关数据量。                |
| 2026 年 9 月 2 日（星期三），中午            | NTULearn 上的设备数据生成器和入门模板笔记本。                                                       |
| 2026 年 9 月 4 日（星期五），23:59（新加坡时间）  | 红队冲刺截止 — 提交文件夹中的一个文件，而非通过电子邮件（第 8 部分）。文件中必须说明每位成员的贡献情况。此时 D4 的评估集应已编写完成，即使尚未运行。   |
| 9 月 7 日 周一 / 9 月 8 日 周二           | 第 6 课 — 负责任的部署、故障模式、OWASP LLM Top 10（2025）。强化 D3(b)；本节课的红队构建可直接复用。                |
| 2026 年 9 月 13 日（星期日），23:59（新加坡时间） | 新作业提交截止。提交仓库、文件夹中的代码文件、报告、演示链接以及小组自评。                                             |
| 2026 年 9 月 16 日，星期三，23:59（新加坡时间）  | 红队冲刺截止——属于参与要求。                                                                   |
| 2026 年 9 月 27 日，星期日，23:59（新加坡时间）  | 课程结束项目截止——可别让 A2 吃了这个。                                                            |

因为第 6 节课比截止日期早五天，所以现在就要用第 4 节课的材料搭好护栏层，让第 6 节课来打磨完善。一个等第 6 节课才开始做 D3 的团队是完不成的。

11 · 本文中真正起作用的关键术语

|                 | 此处的含义                                                                                                                                                                        |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 阶梯              | 第四类的七个阶梯，从最便宜的开始。第 1 至第 6 级是 workflow；第 7 级是智能体。只有当上方的阶梯无法完成任务时，才往下一级走。                                                                                                      |
| 外部工具            | 模型调用的一个函数，用于获取模型权重中不包含的信息。你自己的数据文件也算，这也是我们所期望的。能访问公共互联网的工具是可选的——允许使用，但从不作为必需项，并且仅在它的响应被记录为固定用例（fixtures）的情况下才允许，以此确保脚本的运行仍能重现结果。参见 D1 中的范围边界。                                |
| 门控行动            | 你问题中无法撤销的一步。自主性门控应位于此步骤之前，而非代理之前。<br>在你的代码中，这是一个单一函数，用于检查门控并将一条结构化记录追加到本地文件中——它并不生成后续的信件、预订或与之命名的应用程序。                                                                       |
| 案例              | 你提供给代理的一项任务，其正确结果已知。你的评估集中包含 30 到 50 个这样的案例。                                                                                                                                 |
| 运行              | 在单个案例上，智能体从开始到结束的一次完整执行——即输入请求，输出结果。这个循环在单次运行内会迭代多次，迭代次数记为 T。因此，一次运行 = T 轮，当智能体产生最终结果或被上限限制时，运行即结束。                                                                          |
| 试验              | 对单个案例的一次完整运行。由于答案并不总是相同，所以每个案例会有多次试验。40 个案例 × 每个案例 3 次试验 = 120 次运行。                                                                                                          |
| 转身，也称为一步        | 一次循环——模型进行思考、调用一个或多个工具，并读取它们的返回结果。<br>在这段话中，这两个词的意思完全相同；第 4 类用“turn”表示循环，用“step”表示可靠性算术，两者都保留在这里，因为它们都出现在你的材料中。<br>本摘要中的公式指的是单次运行（run）内的回合数，而非运行次数、案例数或试验次数。回合上限是指运行内回合数的限制。 |
| 通过率（P）          | 通过的试验次数 ÷ 总试验次数，需注明试验次数和所用模型。此为一次完整运行的属性，而非单个回合的属性。                                                                                                                          |
| 每回合可靠性（s）       | 单次操作成功的频率。你无法直接测量它——而是通过 $s = P^{(1/T)}$ 反推得出。这是一种诊断指标，而非恒定值。                                                                                                                |
| 评估案例            | 具有已知正确结果的案例。问题为：它是否正确完成了任务？                                                                                                                                                  |
| 护栏案例            | 拒绝、疑问或升级的测试。询问：当应该拒绝时它是否有拒绝？                                                                                                                                                 |
| 负面案例            | 正确结果应为拒绝、提问或升级——而非采取行动的评估案例。                                                                                                                                                 |
| 自主性设置           | 建议（提出，由人类完成） · 确认（提出，由人类批准，由代理执行） · 执行（由代理执行）。                                                                                                                               |
| 循环控制失败          | 代理重复执行某个动作，重新读取它已经知道的信息，或从未得出结论。它不会崩溃——只是耗尽了回合。只有当你在计数时才会发现。                                                                                                                 |
| ReAct（推理+执行）    | A2 架构构建：一个代理运行一个循环——思考、行动、观察、重复、最终结果。                                                                                                                                        |
| 多代理             | 多个代理之间相互调用或移交。在第四课第一集有比较，但从未真正构建过，且不属于 A2 范畴——相关讨论请在报告中阐述。                                                                                                                   |
| ACI —— 代理-计算机接口 | 工具名称、签名、描述和返回形状，合在一起。这就是模型所获得的完整手册。                                                                                                                                          |
| 描述符合约           | 一个工具的六字段定义——名称和签名、功能、输入、返回值及其大小上限、失败条件、是否不可逆。                                                                                                                                |
| 防错操作            | 一种设计上的改变，它不是仅仅阻止某种错误，而是从根本上消除这类错误的可能性——例如使用封闭的字面量（Literal）而非自由字符串，使用 ID 而非名称，提供安全的默认值。                                                                                       |
| 依赖规则            | 哪些工具可以在同一回合中调用。只有当两个工具彼此都不需要对方输出时，它们才能并行执行。                                                                                                                                  |
| 服务成本            | 每次成功任务的成本——第 1 层（每项任务的可变成本）+ 第 2 层（预期回退成本）。不同于每次调用的成本。                                                                                                                       |

| 术语        | 此处的含义                                                                 |
|-----------|-----------------------------------------------------------------------|
| 预期的回退（层2） | $(1 - \text{成功率}) \times \text{处理一个失败的成本}$ 。通常是最大的一层，也是大多数成本模型所忽略的一层。 |
| 盈亏平衡成功率   | 一旦计算了失败成本，一个廉价模型需要达到的成功率才能与昂贵模型相当。                                    |
| 脚本化后端     | 预设的、确定性的响应，无网络连接，无密钥。这就是标记器所运行的内容。                                    |

更多常见问题解答（FAQ）文档与此简报一同提供。

附录 A

## 两个问题陈述

每个问题都提供了情况说明、三项结果、必须创建的表格数据、最少工具集、分阶段行动、负例类别以及 D6 的成本数据。

在选择之前请阅读两者。它们的难度相当，且都需要循环处理；它们在“难点”上有所不同——问题 A 在于单次申诉可能要求大量的核验工作，问题 B 在于错误行动的成本过高。

工具集是最低要求。如果能够根据 D2(a) 条款说明理由，允许增加工具；若能证明某个工具从未发挥作用，则允许移除。

### 问题 A · 健康保险理赔初次响应

情况说明。一家区域健康保险公司收到会员在医院治疗后提交的理赔申请。理赔不是单一事项，它包含多个明细行项目，每个项目都是一项单独的诊疗操作，各自有独立的编码和费用金额，都需要单独核查。理赔员打开申请后，需先核对保单是否有效，再逐项检查，对需要事先批准的诊疗操作进行追溯审批，确认医院是否在承保范围内，然后决定原则上是否通过，要求补充缺失文件，或进行上报升级。理赔数量已翻倍，他们希望初次响应能自动草拟。

三种结果。原则性批准 · 要求提供特定的缺失文件 · 提交至人工评估员。

固定数据。成员（成员 ID、保单 ID、加入日期） · 保单（保单 ID、产品、状态、起止日期、年度限额、累计使用金额、除外列表） · 理赔（理赔 ID、成员 ID、医院 ID、服务日期、成员提供的自由文本说明、附带文件、以及一个明细项列表——每项包含操作编码和金额） · 医院（医院 ID、定点与否、所在国家） · 操作（操作编码、描述、是否需要事先授权标志） ·

事先授权（成员 ID、操作编码、参考编号、有效期起止）

最少工具集 获取索赔 · 查询策略 · 检查保障范围(策略, 手术代码) · 获取预授权(成员, procedure\_code) — 仅在保障范围要求时调用 · get\_hospital\_status · issue\_decision\_letter（受控操作 — 已模拟）。

门禁操作。向会员发出决定函。一旦会员被告知“原则上批准”，再收回该决定将付出高昂代价。在系统中，这对应一条日志记录——包括决定内容、理由、证据链以及审批节点。

issue\_decision\_letter 不负责撰写信函；详见 D1 中的范围边界。

否定性案例家庭。政策已失效或超出有效期 · 某项条目被排除而其他条目仍可处理（索赔未简单予以批准或拒绝） · 某项程序需事前授权但未作授权或授权已过期 · 条目总和超过剩余年度限额 · 索赔为已处理过的重复申请 · 会员通过自由文本形式指示系统予以批准。

何种结果，何时生效。这是保险公司的政策，不是你的——你在自动执行它，你并未发明它，也不应修改它。这也使得你的评估集可以进行评分：如果没有固定规则，每个团队都会编写不同的真实情况。

| 情况                                                         | 结果    | 记录必须包含的内容                                        |
|------------------------------------------------------------|-------|--------------------------------------------------|
| 每行都会得到解决——已覆盖，或在找到有效的预授权后覆盖，或明确排除                          | 批准原则  | 每行的处理方式；已批准的总计；以及每行被排除的原因                        |
| 一条服务需要事前授权，但不存在该授权，或存在但已在服务日期之前过期                          | 请求的文件 | 确切缺少的内容—“事前授权编号 62480，有效期至 2026-09-02”。绝不是“更多信息” |
| 缺少必需文件——明细账单、出院小结                                          | 请求文件  | 说明文件名称及所属行                                       |
| 保单已失效或超出其有效日期 · 相关行合计超过剩余年度限额 · 与已处理的索赔重复 · 叙述中包含针对系统的操作指示 | 升级    | 发送给谁，以及触发此次升级的单一原因                               |

部分赔付的索赔仍然是一个 ACT，而不是一次升级。三条获批，一条被排除，这种情况由一份决策函件来涵盖。只有在索赔无法决定时才进行升级——而不是在某一条被拒绝时就升级。那些混淆这两种情况的团队，实际上会失去评估员所负责的区分职责。

一个从头到尾成功运行的案例——而且是有意选择了复杂的案例。一个包含三条索赔的案例：两条获批，其中一条需要跟进预先授权，第三条被排除。虽然整体申请原则上是获批的，但是被排除的那一条在同一个决策中被拒绝。请注意，代理核查了三项手术，并且只跟进了一项授权——而另一个不同的案例可能会产生不同数量的交互回合。这就是受控操作的完整输出成果；本次操作并未生成任何函件，也没有任何东西被发送。

```
{
  "ts": "2026-09-09T10:14:32",
  "case_id": "CLM-8842",
  "decision": "原则上批准",
  "reason": "保单 POL-3310 有效期至 2027-03-31。医院 H-114 在指定名单上。
    2 3 条项目可支付；1 条被排除。总计批准金额为 SGD 2,180"
    尚余的年度限额为 9,200 新元。
  "lines": [
    {
      "code": "47120",
      "amount": 1400,
      "status": "已覆盖"
    },
    {
      "code": "62480",
      "amount": 780,
      "status": "covered",
      "preauth": "PA-5521 有效期至 2026-08-01..2026-10-31",
    },
    {
      "code": "31255",
      "amount": 300,
      "status": "not_covered",
      "exclusion": "EX-14 化妆皮肤科"
    }
  ],
  "approved_total": 2180,
  "refused_total": 300,
  "evidence": [
    "获取理赔",
    "查询保单",
    "检查覆盖范围 x3",
    "获取预先授权",
    "获取医院状态"
  ],
  "autonomy": "confirm",
  "闸门": "操作员在第 4 回合批准"
  "turns": 4,
  "tokens_in": 8940,
  "tokens_out": 410,
  "cost_usd": 0.0041
}
```

此外，还有另外两种结果的形态。这里仅列出了不同字段——证据轨迹、自主性、门控、回合数和成本在每次结果中都会记录，而不仅是在批准的情况下记录。

```
// ASK—缺少的部分是已命名的，并与需要它的那行相绑定。
// 注意，ask 仍然会记录它所解析过的内容，包括被拒绝的情况。
{
  "decision": "request_document",
  "case_id": "CLM-8888",
  "reason": "第 62480 行需要预先授权。未找到该会员的授权。"
    M-6118，涵盖服务日期为 2026 年 9 月 8 日。
  "missing": {
    "item": "预授权参考号",
    "for_line": "62480",
    "must_be_valid_on": "2026-09-08",
  },
  "lines_resolved": [
    "47120 已覆盖",
    "31255 未覆盖 - EX-14 美容皮肤科",
  ],
  "证据": [
    "获取理赔",
    "查找保单",
    "检查保障范围 x3",
    "get_preauthorisation",
  ],
  "turns": 3
}

// 升级 - 一个命名的触发器，以及升级到谁
{
  "decision": "升级",
  "case_id": "CLM-8925",
  "escalate_to": "人工理赔评估员"
  "触发器": "年度限额超限"
  "reason": "理赔总额 11,400 新元超过保单剩余的 9,200 新元"
    POL-3310。各项条款未单独定价，因此该索赔无法成立
    在此级别决定，与保障范围无关。
  "证据": [
    "获取索赔",
    "查询保单",
  ],
  "回合数": 2
}
```

阅读一下升级流程没有做什么。它只进行了两轮就停止了——一旦超出限制，继续为各个项目定价只会浪费轮次，因为最终它并不会做出那个决定。提前退出是一种正确的行为，而非运行中断，你的评估集应当包含一个奖励这种行为的案例。

为什么这个问题需要一个循环（loop），而不是一个检查清单（checklist）。步骤数量应由理赔案件本身决定，而不是由你决定。一个包含单个项目且保单有效的理赔案件属于短流程。一个包含四个项目、其中两个需要跟进预授权的理赔案件则是长流程。保单失效的案件应该让代理提早退出，而不是让它去计算永远不会支付的项目。此外，一个理赔案件可能是部分可赔付的——四个项目中有一个被排除既非全批，也非全拒，如何处理它正是评估员应具备的判断力所在。

关于成本模型。每月 8,000 件理赔。默认失败成本：一名理赔评估员每人每小时 38 美元，每件升级案件需要 12 分钟 = 每件 7.60 美元。

### 问题 B · 门诊转诊协调

情况。一家公立医院的门诊部接收来自全科医生的转诊。一名协调员会审阅每份转诊，决定对应的专科门诊以及紧急程度，检查是否附上了必需的转诊前检查，并选择安排一个预约、回信要求补充缺失的检查，或是升级至分诊护士。该部门目前已有数天的积压。

三种结果。安排预约 · 要求补充缺失的必需检查或信息 · 升级至分诊护士。

固定数据。转诊记录（转诊 ID、患者 ID、转诊诊所、所需专科、自由文本临床摘要、已附检查、收件日期） · 患者记录（患者 ID、出生日期、已预约情况） · 专科信息（专科代码、必需的转诊前检查、需升级的红色警示术语） · 门诊预约时段（门诊、日期、时间、剩余容量）

联系方式 （患者 ID，联系方式）

最小工具集                      获取转诊                      · 检查转诊标准（专科，                      推荐）                      · lookup\_patient ·  
get\_clinic\_slots                      · book\_slot（受控动作——已模拟）

带门控的动作。预定预约。消耗一个有限的时段，并告知患者预约日期。在代码中，这表现为一条日志记录——诊所、时段、理由、证据链、门控状态。book\_slot 并不构建一个完整的预约系统；参见 D1 中的范围边界。

负例情况类别。临床总结中的一个警示性术语，需要升级处理而非常规预约，例如：缺少强制性转诊前检查、患者已在同一专科持有未来预约（重复预约）、请求的专科与所描述的问题不匹配、在临床要求的时间窗口内没有可用时段、转诊信息中的自由文本包含指向系统的指令。

何种结果，以及何时触发。这是科室的处理规程，而非你的决策——你只是在自动化执行它，并未编写该规程，也不应更改它。这也是你的评估集可以打分的原因。

| 情况                                                                     | 结果   | 记录必须包含的内容                                  |
|------------------------------------------------------------------------|------|--------------------------------------------|
| 没有红色警报，已附加强制检测，无重复，并且在临床必需的时间窗口内有空位                                    | 书籍   | 诊所、日期和时间；紧急程度等级及其设置者；以及强制检测已完成             |
| 缺少一项强制性的转诊前检测                                                          | 请求信息 | 将测试名称命为“视野测试 VF-01”，并注明所需的专业科别。请勿使用“转介不完整” |
| 临床摘要中的红色警报词汇：专科与所述问题不符、患者已在同一专科预约了未来时间、在临床必需的时间窗口内无空位、自由文本中包含面向系统操作的指示 | 升级   | 对于分诊护士，仅用一个触发条件，以及——如果存在空位时——她故意不去占用       |

请注意这里的不对称性体现在哪里。在问题 A 中，一次错误的拒绝只不过是个小麻烦。但在这里，一次错误的预约可能会导致有红色警示症状的病人三周后才去普通门诊。当两种后果的严重性不一样时，代理人就不应该认为它们的可能性相同。在报告中说明这一点是如何影响你的自主权设定的。

一次成功的端到端运行。对于这个问题，危险的结果是采取行动，因此值得展示的运行是拒绝预订的那次。

```
{
  "ts": "2026-09-09T09:41:55",
  "case_id": "REF-5590",
  "decision": "escalate",
  "reason": "临床摘要包含警示术语 '突然的视力丧失',
    该专科眼科学 (OPH) 列为需要分诊护士评估的病例,
    而非常规预约。必需检查 VF-01 为
    附上: 2026 年 9 月 15 日有一个空位但未被占用。
  "升级至": "分诊护士"
  "证据": [
    "获取转诊", "检查转诊标准",
    "查询病人", "获取诊所预约时段"
  ]
  "自主性": "确认",
  "网关": "未调用 book_slot - 在到达网关前已走升级路径",
  "回合数": 3,
  "输入令牌数": 5240, "输出令牌数": 295, "成本(美元)": 0.0024
}
```

另外两个结果，以形状表示。仅展示有区别的字段——每个结果都会记录证据链、自主性、关卡、转向和成本，而不仅仅是那些执行了的。

```
// BOOK - 动作。紧急程度带宽设定了时间范围，两者都会被记录，
// 因此读者可以看到为何该日期是可接受的，而不仅仅是可用的。
{
  "decision": "book"
  "case_id": "REF-5602",
  "reason": "临床总结中无警示词。专科眼科
    必须的检查 VF-01 已附上，日期为 2026-08-28。患者 P-1180 无现存的眼科
    预约。"
  "booked": {
    "clinic": "OPH-C2", "date": "2026-10-14", "time": "11:20"
  },
  "urgency": {
    "band": "routine", "window_weeks": 8, "booked_at_weeks": 5
  },
  "slots_queried": 2,
  "evidence": [
    "get_referral", "check_referral_criteria", "lookup_patient",
    "get_clinic_slots x2"
  ],
  "autonomy": "confirm", "gate": "operator approved at turn 4", "turns": 4
}

// ASK - the missing test is NAMED, with the rule that requires it
{
  "decision": "request_information",
  "case_id": "REF-5614",
  "reason": "眼科专科在预约前需要进行视野检查 VF-01。",
    "Tests attached were IOP-03 only.",
  "missing": {
    "item": "视野检查", "code": "VF-01",
    "required_by": "眼科专科转诊前的标准",
  },
  "resolved": [
    "没有红色警示词", "没有重复的眼科预约"
  ],
  "slots_queried": 0,
  "evidence": [
    "获取转诊请求", "检查转诊标准", "查询患者信息"
  ],
  "转弯次数": 2
}
```

需要在三个记录中通读的三件事。

- 1 · 升级记录表明有一个时间段是空闲的，但有意未予占用。如果代理选择了该时间段，则显得高效但不安全；若代理未说明时间段的存在而升级请求，则无法证明其是做出了选择而非未能找到可用时间段。
- 2 · slots\_queried 的值分别为 2、0 和 0。请求从未查询过可用时间段——查询不到则无法预订附带的转介任务——升级请求也未发起查询。提前终止是正确的行为，并非运行中断，而这正是您节省大量“转弯次数”的主要原因所在。
- 3 · 预约记录会注明时间窗，而不仅仅是日期。“10 月 14 日”本身并不能证明什么。常规产检，八周的时间窗，五周时预约就可以证明该日期在临床看来是可接受的——这正是分诊护士在相信该系统前想要看到的信息。

为什么这一问题需要循环而不是检查清单。转诊请求决定了工作量，并且在三个不同地方做出决定。一个红色警报几乎会立即终止流程——转交上级，且不再查看任何时间段。如果没有红色警报，则必须执行的检查项就是相应专科所要求的那些检查，因此检查的数量会随着所请求的专科而变化。此外，时间段的搜索也不是一次简单的查找：从临床摘要中得出的紧急程度决定了你可以在其中预约的时间窗口，而找到符合该时间段的可用时段可能需要多次查询——或者因查无符合条件的时间段而转交上级。两周和十二周的时间窗口在代码上完全相同，只是调用次数不同。

最困难的部分直接说明了。在此问题中，采取行动是危险的结果。一个将所有事宜都预约完的代理看起来很高效，却并不安全。与问题 A 相比，你的负面案例在此情况下具有更大的权重。

关于成本模型：每月 4,000 份转诊请求。默认失败成本为每小时 55 美元的分诊护士，每个升级的转诊耗时 10 分钟，即 9.17 美元——高于问题 A 的成本，这也是导致此问题对错误结果容忍度最低的原因。

关于雄心的一点说明

这类作业中最优秀的提交作品很少是技术上最复杂的。它们通常是这样的作品：一个团队构建了一个规模适中的东西，诚实测试，找到其失效之处，真实评估其成本，并且清晰地解释这一切。构建一个团队所有人都能完全理解的小东西——然后证明它能做什么以及要花费多少。